Spring Rest API

What is Spring REST API?

A **Spring REST API** is a way of building web services using the Spring framework that follow **REST (Representational State Transfer)** principles. It allows applications to communicate over HTTP by exposing endpoints that clients (like browsers, mobile apps, or other services) can call. These APIs typically exchange data in formats like JSON or XML, making them lightweight and easy to integrate across platforms.

In simple terms, a Spring REST API acts as a **bridge between frontend and backend**, where the backend processes requests and returns structured data instead of HTML pages. It is widely used in modern applications such as mobile apps, microservices, and single-page applications (SPA).

What is REST Architecture?

REST is an architectural style that defines how web services should behave. It is based on **resources**, where each resource is identified by a unique URL, and operations are performed using HTTP methods like GET, POST, PUT, and DELETE. REST is stateless, meaning each request from the client must contain all the information needed to process it.

The key idea of REST is simplicity and scalability. Instead of maintaining sessions, the server treats every request independently, which improves performance and makes it easier to scale applications. REST also promotes a uniform interface, making APIs predictable and easy to use.

Key Characteristics of REST

REST APIs follow certain important principles that define their behavior. First, they are **stateless**, meaning no client data is stored on the server between requests. Second, they use a **client-server architecture**, separating frontend and backend responsibilities.

Another important characteristic is **uniform interface**, where standard HTTP methods are used consistently. REST APIs are also **cacheable**, meaning responses can be stored and reused to improve performance. These features

make REST APIs efficient, scalable, and widely adopted in modern software development.

Spring Boot and REST API

Spring Boot simplifies the creation of REST APIs by reducing configuration and setup. It provides built-in support for creating web applications using annotations like @RestController and @RequestMapping. With Spring Boot, developers can quickly build production-ready APIs without worrying about complex configurations.

Spring Boot also includes an embedded server like Tomcat, so you don't need to deploy your application separately. This makes development faster and easier, especially for beginners. It also integrates seamlessly with databases, security, and other Spring modules.

Important Annotations in Spring REST

Spring REST APIs rely heavily on annotations to define behavior. The @RestController annotation is used to mark a class as a REST controller, combining @Controller and @ResponseBody. It ensures that methods return data directly instead of views.

Other important annotations include @GetMapping, @PostMapping, @PutMapping, and @DeleteMapping, which map HTTP methods to Java methods. @RequestBody is used to convert JSON data into Java objects, while @PathVariable and @RequestParam help extract values from the URL.

HTTP Methods in REST API

REST APIs use standard HTTP methods to perform operations on resources. The **GET** method is used to retrieve data from the server, while **POST** is used to create new resources. The **PUT** method updates existing data, and **DELETE** removes resources.

Each method has a specific purpose, making APIs intuitive and easy to understand. For example, a GET request to /users retrieves all users, while a

POST request to the same endpoint creates a new user. This consistency is a key advantage of REST APIs.

Request and Response Handling

In a Spring REST API, a client sends a request to a specific endpoint, and the server processes it and returns a response. The request may include headers, parameters, and a body containing data. Spring automatically converts JSON data into Java objects using libraries like Jackson.

The response typically includes a status code (like 200, 404, or 500) and a body containing the requested data. Spring provides classes like ResponseEntity to customize responses, including headers and status codes. This makes API responses flexible and informative.

JSON and Data Representation

JSON (JavaScript Object Notation) is the most commonly used format in REST APIs. It is lightweight, human-readable, and easy for machines to parse. Spring uses Jackson to automatically convert Java objects into JSON and vice versa.

For example, a Java object representing a user can be returned as JSON in the response. This allows frontend applications to easily consume and display the data. JSON plays a crucial role in enabling communication between different systems.

Exception Handling in REST API

Handling errors properly is important in REST APIs. Spring provides mechanisms like @ExceptionHandler and @ControllerAdvice to manage exceptions globally. This ensures that errors are returned in a consistent and meaningful format.

Instead of exposing internal errors, APIs can return structured error responses with proper status codes. For example, if a user is not found, the API can return a 404 status with a clear message. This improves the user experience and debugging process.

Validation in Spring REST

Validation ensures that the data received from clients is correct and meets certain criteria. Spring provides validation annotations like @NotNull, @Size, and @Email to enforce rules on input data. These validations are applied automatically when using @Valid.

If validation fails, Spring returns an error response indicating what went wrong. This prevents invalid data from entering the system and ensures data integrity. Validation is especially important in forms like registration and login APIs.

Database Integration

Spring REST APIs often interact with databases to store and retrieve data. This is typically done using Spring Data JPA or JDBC. Developers can define repository interfaces to perform CRUD operations without writing complex SQL queries.

Spring Data JPA also supports custom queries and native queries for advanced use cases. It simplifies database interaction and reduces boilerplate code. This makes it easier to build scalable and maintainable applications.

Security in Spring REST API

Security is a critical aspect of REST APIs. Spring Security provides features like authentication and authorization to protect endpoints. APIs can use mechanisms like JWT (JSON Web Token) to secure communication between client and server.

With proper security, only authorized users can access certain resources. For example, an admin can access all data, while a normal user can access only their own data. Security ensures data protection and prevents unauthorized access.

Real-World Example

Consider a user management system where a frontend application communicates with a Spring REST API. The API provides endpoints like /users

to fetch users, /users/{id} to get a specific user, and /users (POST) to create a new user.

The frontend sends requests, and the API processes them, interacts with the database, and returns responses in JSON format. This architecture is commonly used in applications like e-commerce platforms, banking systems, and social media apps.

Advantages of Spring REST API

Spring REST APIs offer several advantages, including simplicity, scalability, and flexibility. They are easy to develop and maintain due to Spring Boot's auto-configuration and annotation-based approach. They also support integration with various tools and technologies.

Another major advantage is platform independence, as REST APIs can be consumed by any client that supports HTTP. This makes them ideal for building modern distributed systems and microservices architectures.

Conclusion

Spring REST API is a powerful way to build modern web services that are scalable, efficient, and easy to use. By following REST principles and leveraging Spring Boot features, developers can create robust APIs with minimal effort.

Understanding concepts like HTTP methods, annotations, validation, exception handling, and security is essential for mastering Spring REST APIs. Once you are comfortable with these topics, you can build complete real-world applications with confidence.

1. Project Structure

com.example.studentapi
│
├── controller
│ └── StudentController.java
├── service
│ └── StudentService.java

```
├── repository
│   └── StudentRepository.java
├── entity
│   └── Student.java
├── exception
│   ├── StudentNotFoundException.java
│   └── GlobalExceptionHandler.java
└── StudentApiApplication.java
```

2. Student Entity (Model)

This class represents the **data structure** and uses validation annotations to ensure valid input.

```
package com.example.studentapi.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name cannot be empty")
    private String name;

    @Email(message = "Invalid email format")
    private String email;

    @Min(value = 18, message = "Age must be >= 18")
    private int age;
```

```
    public Student() {}

    public Student(Long id, String name, String email, int age) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.age = age;
    }

    // Getters and Setters
}
```

Explanation

This class is mapped to a database table using @Entity. Validation annotations like @NotBlank, @Email, and @Min ensure only valid data is accepted. When a request comes with invalid data, Spring automatically throws validation errors.

---

3. Repository Layer

This interface handles database operations.

```
package com.example.studentapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.studentapi.entity.Student;

public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

Explanation

By extending JpaRepository, we get all CRUD operations automatically like save(), findById(), findAll(), and deleteById(). No need to write SQL queries manually.

---

4. Custom Exception

```
package com.example.studentapi.exception;

public class StudentNotFoundException extends RuntimeException {
   public StudentNotFoundException(String message) {
      super(message);
   }
}
```

Explanation

This exception is thrown when a student is not found in the database. It helps us provide meaningful error messages instead of generic errors.

---

5. Service Layer

This contains business logic.

```
package com.example.studentapi.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

import com.example.studentapi.entity.Student;
import com.example.studentapi.repository.StudentRepository;
import com.example.studentapi.exception.StudentNotFoundException;

@Service
public class StudentService {

   @Autowired
   private StudentRepository repo;

   public Student saveStudent(Student student) {
      return repo.save(student);
   }

   public List<Student> getAllStudents() {
      return repo.findAll();
```

```
    }

    public Student getStudentById(Long id) {
        return repo.findById(id)
                .orElseThrow(() -> new StudentNotFoundException("Student not
found with id " + id));
    }

    public Student updateStudent(Long id, Student student) {
        Student existing = getStudentById(id);
        existing.setName(student.getName());
        existing.setEmail(student.getEmail());
        existing.setAge(student.getAge());
        return repo.save(existing);
    }

    public void deleteStudent(Long id) {
        Student student = getStudentById(id);
        repo.delete(student);
    }
}
```

Explanation

The service layer separates business logic from the controller. It also throws StudentNotFoundException when data is not found, ensuring proper error handling.

---

6. Controller Layer (REST API)

```
package com.example.studentapi.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import jakarta.validation.Valid;
```

```
import java.util.List;

import com.example.studentapi.entity.Student;
import com.example.studentapi.service.StudentService;

@RestController
@RequestMapping("/students")
public class StudentController {

    @Autowired
    private StudentService service;

    @PostMapping
    public ResponseEntity<Student> createStudent(@Valid @RequestBody
Student student) {
        return ResponseEntity.ok(service.saveStudent(student));
    }

    @GetMapping
    public ResponseEntity<List<Student>> getAllStudents() {
        return ResponseEntity.ok(service.getAllStudents());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getStudent(@PathVariable Long id) {
        return ResponseEntity.ok(service.getStudentById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Student> updateStudent(@PathVariable Long id,
                                    @Valid @RequestBody Student student) {
        return ResponseEntity.ok(service.updateStudent(id, student));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<String> deleteStudent(@PathVariable Long id) {
        service.deleteStudent(id);
```

```
        return ResponseEntity.ok("Student deleted successfully");
    }
}
```

Explanation

The controller handles HTTP requests and returns responses using ResponseEntity. The @Valid annotation ensures validation is applied before processing the request. Each method corresponds to REST operations like GET, POST, PUT, and DELETE.

---

7. Global Exception Handler (@ControllerAdvice)

```
package com.example.studentapi.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.Map;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<String>
handleStudentNotFound(StudentNotFoundException ex) {
        return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationException(
            MethodArgumentNotValidException ex) {

        Map<String, String> errors = new HashMap<>();
```

```
        ex.getBindingResult().getFieldErrors().forEach(error ->
                errors.put(error.getField(), error.getDefaultMessage())
        );

        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleGlobalException(Exception ex) {
        return new ResponseEntity<>("Something went wrong: " +
ex.getMessage(),
                HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Explanation

This class handles all exceptions globally using @ControllerAdvice. It ensures consistent error responses across the application. Validation errors return field-wise messages, while custom exceptions return meaningful responses.

---

8. Main Application Class

```
package com.example.studentapi;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentApiApplication.class, args);
    }
}
```

When does MethodArgumentNotValidException get triggered?

MethodArgumentNotValidException is triggered **when validation on a request body fails** in a Spring REST API. This happens specifically when you use @Valid (or @Validated) on a method parameter and the incoming data violates the validation rules defined in your model class.

In simple terms, when the client sends invalid data (like empty name, wrong email, or age less than allowed), Spring automatically checks the constraints. If any constraint fails, it does **not call your controller method logic** and instead throws MethodArgumentNotValidException.

Exact Flow of Execution

When a request comes to your controller, Spring follows a sequence. First, it converts JSON into a Java object using Jackson. After that, it checks validation annotations like @NotBlank, @Email, and @Min if @Valid is present.

If all validations pass, the controller method executes normally. But if **any validation fails**, Spring immediately throws MethodArgumentNotValidException. This exception is then handled by your @ControllerAdvice class instead of executing the controller logic.

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>> handleValidationException(
MethodArgumentNotValidException ex) {

Map<String, String> errors = new HashMap<>();

List<FieldError> fieldErrors = ex.getBindingResult().getFieldErrors();

for (FieldError error : fieldErrors) {
String fieldName = error.getField();
String errorMessage = error.getDefaultMessage();
errors.put(fieldName, errorMessage);
}
```

```
return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
}
```

What are HTTP Status Codes?

HTTP status codes are **standard responses sent by the server** to indicate the result of a client's request. When a client (like a browser or Postman) sends a request to a REST API, the server processes it and returns a status code along with the response.

These codes help the client understand whether the request was successful, failed, or needs further action. In Spring REST APIs, status codes are usually returned using ResponseEntity, which allows full control over the response.

Categories of Status Codes

HTTP status codes are divided into five categories based on their first digit. Each category represents a different type of response from the server.

1xx – Informational

These codes indicate that the request has been received and is being processed. They are rarely used in REST APIs and are mostly internal to HTTP communication.

2xx – Success

These codes mean the request was successfully processed. They are the most commonly used in REST APIs when operations complete without issues.

3xx – Redirection

These indicate that the client needs to take additional action, such as redirecting to another URL. These are mostly used in web applications, not much in REST APIs.

4xx – Client Errors

These occur when the client sends invalid or incorrect data. It means the problem is on the client side.

5xx – Server Errors

These indicate that something went wrong on the server side, even though the request might be valid.

Important Status Codes and Their Meaning

200 OK

This means the request was successful and the server is returning the requested data. It is commonly used for GET requests and successful updates.

👉 Use when: Data is fetched or updated successfully.

---

201 Created

This indicates that a new resource has been successfully created.

👉 Use when: A new record is inserted into the database (POST request).

---

204 No Content

This means the request was successful, but there is no content to return.

👉 Use when: Delete operation is successful and no response body is needed.

---

400 Bad Request

This indicates that the client sent invalid data or a malformed request.

👉 Use when: Validation fails (@Valid errors).

---

401 Unauthorized

This means authentication is required or failed.

👉 Use when: User is not logged in or token is invalid.

---

403 Forbidden

This means the user is authenticated but does not have permission.

👉 Use when: User tries to access restricted resources.

---

404 Not Found

This indicates that the requested resource does not exist.

👉 Use when: Student ID is not found in the database.

---

500 Internal Server Error

This means something went wrong on the server.

👉 Use when: Unexpected exceptions occur.

Examples in Student REST API

1. Create Student (POST → 201 Created)

```
@PostMapping
public ResponseEntity<Student> createStudent(@Valid @RequestBody
Student student) {
   Student saved = service.saveStudent(student);
   return new ResponseEntity<>(saved, HttpStatus.CREATED);
}
```

---

2. Get Student (GET → 200 OK)

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudent(@PathVariable Long id) {
   return new ResponseEntity<>(service.getStudentById(id), HttpStatus.OK);
}
```

---

3. Delete Student (DELETE → 204 No Content)

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteStudent(@PathVariable Long id) {
   service.deleteStudent(id);
   return new ResponseEntity<>(HttpStatus.NO_CONTENT);
}
```

---

4. Validation Error (400 Bad Request)

Handled in @ControllerAdvice:

```
return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
```

---

5. Not Found (404)

```
@ExceptionHandler(StudentNotFoundException.class)
public ResponseEntity<String> handleNotFound(StudentNotFoundException
ex) {
   return new ResponseEntity<>(ex.getMessage(), HttpStatus.NOT_FOUND);
}
```

---

6. Server Error (500)

```
@ExceptionHandler(Exception.class)
public ResponseEntity<String> handleGlobalException(Exception ex) {
   return new ResponseEntity<>("Error occurred",
HttpStatus.INTERNAL_SERVER_ERROR);
}
```

Shortcut Methods of ResponseEntity (Important)

Spring provides shorter ways to write responses:

```
return ResponseEntity.ok(data);              // 200
return ResponseEntity.status(HttpStatus.CREATED).body(data); // 201
return ResponseEntity.noContent().build();   // 204
return ResponseEntity.badRequest().body(error); // 400
```